7
Basics of Object Detection

In the previous chapters, we learned about performing image classification. Imagine a scenario where we leverage computer vision for a self-driving car. It is not only necessary to detect whether the image of a road contains images of vehicles, a sidewalk, and pedestrians, but it is also important to identify *where* those objects are located. The various techniques of object detection that we will study in this chapter and the next will come in handy in such a scenario.

In this chapter and the next, we will learn about some of the techniques for performing object detection. We will start by learning the fundamentals – labeling the ground truth bounding-box of objects within an image using a tool named `ybat`, extracting region proposals using the `selectivesearch` method, and defining the accuracy of bounding-box predictions by using the **intersection over union** (**IoU**) and mean average precision metrics. After this, we will learn about two region proposal-based networks – R-CNN and Fast R-CNN – by first learning about their working details and then implementing them on a dataset that contains images belonging to trucks and buses.

The following topics will be covered in this chapter:

- Introducing object detection
- Creating a bounding-box ground truth for training
- Understanding region proposals
- Understanding IoU, non-max suppression, and mean average precision
- Training R-CNN-based custom object detectors
- Training Fast R-CNN-based custom object detectors

All code snippets within this chapter are available in the `Chapter07` folder of the GitHub repository at https://bit.ly/mcvp-2e.

Introducing object detection

With the rise of autonomous cars, facial detection, smart video surveillance, and people-counting solutions, fast and accurate object detection systems are in great demand. These systems include not only object classification from an image but also the location of each one of the objects, by drawing appropriate bounding boxes around them. This (drawing bounding boxes and classification) makes object detection a harder task than its traditional computer vision predecessor, image classification.

Before we explore the broad use cases of object detection, let's understand how it adds to the object classification task that we covered in the previous chapter. Imagine a scenario where you have multiple objects in an image. I ask you to predict the class of objects present in the image. For example, let's say that the image contains both cats and dogs. How would you classify such images? Object detection comes in handy in such a scenario, where it not only predicts the location of objects (bounding box) present in it but also predicts the class of the objects present within the individual bounding boxes.

To understand what the output of object detection looks like, let's go through the following figure:

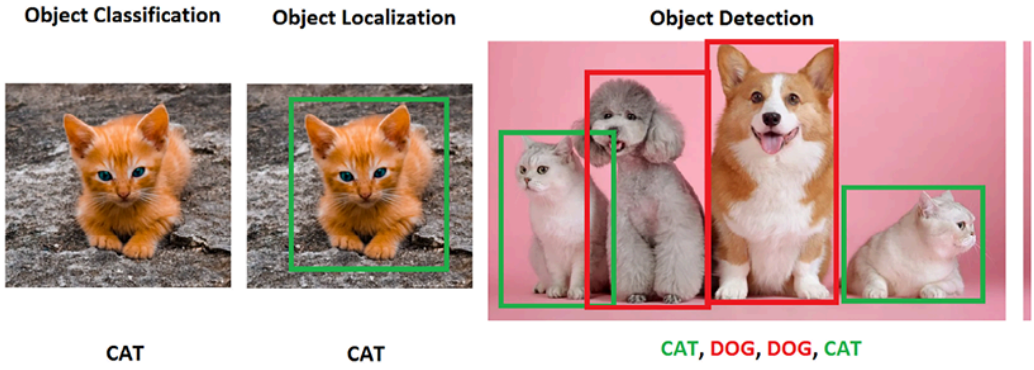


Figure 7.1: Distinction between object classification and detection

In the preceding figure, we can see that, while a typical object classification merely mentions the class of object present in the image, object localization draws a bounding box around the objects present in the image. Object detection, on the other hand, would involve drawing the bounding boxes around individual objects in the image, along with identifying the class of object within a bounding box across the multiple objects present in the image.

Some of the various use cases leveraging object detection include the following:

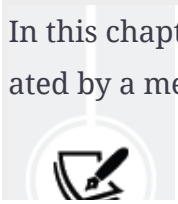
- **Security**: This can be useful for recognizing intruders.
- **Autonomous cars**: This can be helpful in recognizing the various objects present in the image of a road.
- **Image searching**: This can help identify the images containing an object (or a person) of interest.
- **Automotives**: This can help in identifying a number plate within the image of a car.

In all the preceding cases, object detection is leveraged to draw bounding boxes around a variety of objects present within the image.

In this chapter, we will learn about predicting the class of the object and having a tight bounding box around the object in the image, which is the localization task. We will also learn about detecting the class corresponding to multiple objects in the picture, along with a bounding box around each object, which is the object detection task.

Training a typical object detection model involves the following steps:

1. Creating ground-truth data that contains labels of the bounding box and class corresponding to various objects present in the image
2. Coming up with mechanisms that scan through the image to identify regions (region proposals) that are likely to contain objects



In this chapter, we will learn about leveraging region proposals generated by a method named `SelectiveSearch`. In the next chapter, we will

learn about leveraging anchor boxes to identify regions containing objects.

3. Creating the target class variable by using the IoU metric
4. Creating the target bounding-box offset variable to make corrections to the location of the region proposal in *step 2*
5. Building a model that can predict the class of object along with the target bounding-box offset corresponding to the region proposal
6. Measuring the accuracy of object detection using **mean average precision (mAP)**

Now that we have a high-level overview of what is to be done to train an object detection model, we will learn about creating the dataset for a bounding box (which is the first step in building an object detection model) in the next section.

Creating a bounding-box ground truth for training

We have learned that object detection gives us output in the form of a bounding box surrounding the object of interest in an image. For us to build an algorithm that detects this bounding box, we would have to create input-output combinations, where the input is the image and the output is the bounding boxes and the object classes.

Note that when we detect the bounding box, we are detecting the pixel locations of the four corners of the bounding box surrounding the image.

To train a model that provides the bounding box, we need the image and the corresponding bounding-box coordinates of all the objects in the image. In this section, we will learn one way to create the training dataset, where the image is the input and the corresponding bounding boxes and classes of objects are stored in an XML file as output.

Here, we will install and use `ybat` to create (annotate) bounding boxes around objects in the image. We will also inspect the XML files that contain the annotated class and bounding-box information.

Note that there are alternative image annotation tools like CVAT and Label Studio.

Let's start by downloading `ybat-master.zip` from GitHub (**https://github.com/drainingsun/ybat**) and unzipping it. Then, open `ybat.html` using a browser of your choice.

Before we start creating the ground truth corresponding to an image, let's specify all the possible classes that we want to label across images and store in the `classes.txt` file, as follows:

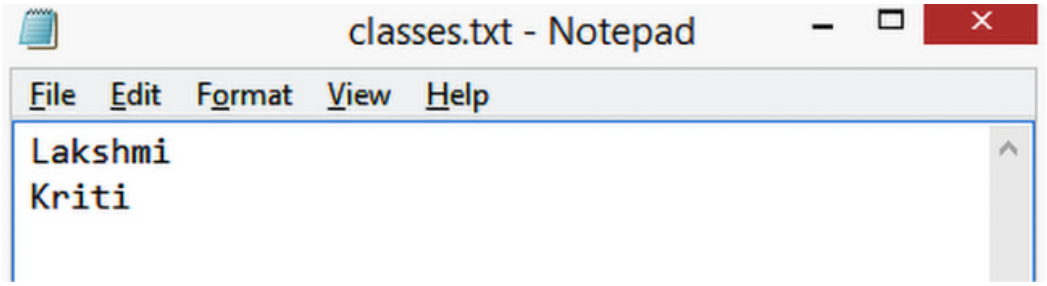


Figure 7.2: Providing class names

Now, let's prepare the ground truth corresponding to an image. This involves drawing a bounding box around objects (the persons in the following figure) and assigning labels/classes to the objects present in the image, as seen in the following steps:

1. Upload all the images you want to annotate.
2. Upload the `classes.txt` file.
3. Label each image by first selecting the filename and then drawing a crosshair around each object you want to label. Before drawing a crosshair, ensure you select the correct class in the `classes` region (the `classes` pane can be seen below *step 2* in the following image).
4. Save the data dump in the desired format. Each format was independently developed by a different research team, and all are equally

valid. Based on their popularity and convenience, every implementation prefers a different format.

As you can see, the preceding steps are represented in the following figure:

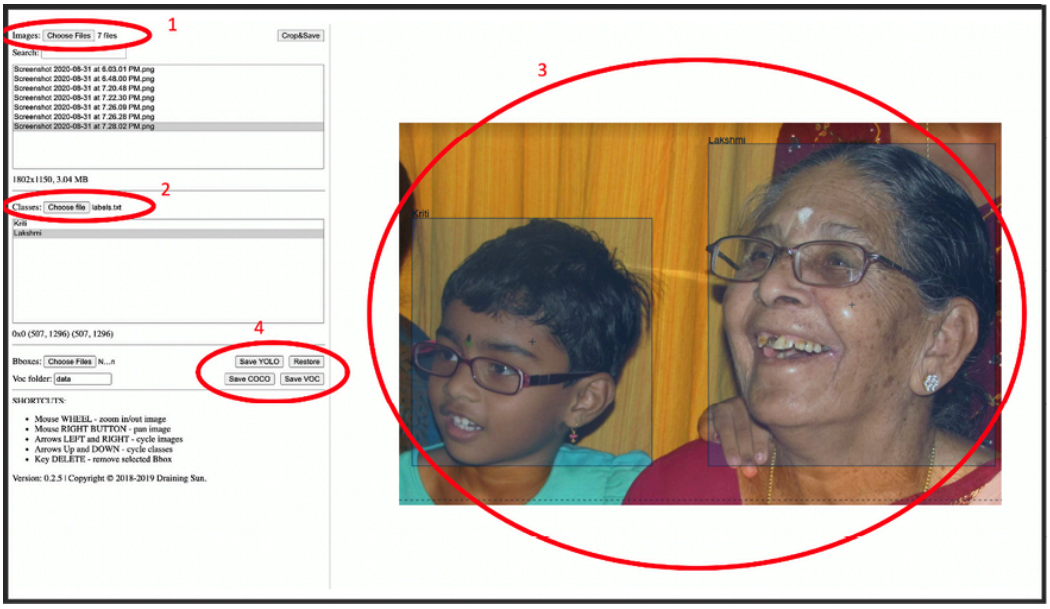


Figure 7.3: Annotation steps

For example, when we download the PascalVOC format, it downloads a zip of XML files. A snapshot of the XML file after drawing the rectangular bounding box is available on GitHub as `sample_xml_file. xml`. There, you will observe that the `bndbox` field contains the coordinates of the minimum and maximum values of the *x* and *y* coordinates, corresponding to the objects of interest in the image. We should also be able to extract the classes corresponding to the objects in the image using the `name` field.

Now that we understand how to create a ground truth of objects (a class label and bounding box) present in an image, let's dive into the building blocks of recognizing objects in an image. First, we will go through region proposals that help to highlight the portions of the image that are most likely to contain an object.

Understanding region proposals

Imagine a hypothetical scenario where the image of interest contains a person and sky in the background. Let's assume there is little change in the pixel intensity of the background (sky) and a considerable change in the pixel intensity of the foreground (the person).

Just from the preceding description itself, we can conclude that there are two primary regions here – the person and the sky. Furthermore, within the region of the image of a person, the pixels corresponding to hair will have a different intensity to the pixels corresponding to the face, establishing that there can be multiple sub-regions within a region.

Region proposal is a technique that helps identify islands of regions where the pixels are similar to one another. Generating a region proposal comes in handy for object detection where we must identify the locations of objects present in an image. Additionally, given that a region proposal generates a proposal for a region, it aids in object localization where the task is to identify a bounding box that fits exactly around an object. We will learn how region proposals assist in object localization and detection in a later section, *Training R-CNN-based custom object detectors*, but let's first understand how to generate region proposals from an image.

Leveraging SelectiveSearch to generate region proposals

SelectiveSearch is a region proposal algorithm used for object localization, where it generates proposals of regions that are likely to be grouped together based on their pixel intensities. SelectiveSearch groups pixels based on the hierarchical grouping of similar pixels, which, in turn, leverages the color, texture, size, and shape compatibility of content within an image.

Initially, SelectiveSearch over-segments an image by grouping pixels based on the preceding attributes. Then, it iterates through these over-segmented groups and groups them based on similarity. At each iteration, it combines smaller regions to form a larger region.

Let's understand the `selectivesearch` process through the following example:

Find the full code for this exercise at `Understanding_selectivesearch.ipynb` in the `Chapter07` folder of this book's GitHub repository at **https://bit.ly/mcvp-2e**.

1. Install the required packages:

```
%pip install selectivesearch
%pip install torch_snippets
from torch_snippets import *
import selectivesearch
from skimage.segmentation import felzenszwalb
```

2. Fetch and load the required image:

```
!wget https://www.dropbox.com/s/l98leemr7/Hemanvi.jpeg
img = read('Hemanvi.jpeg', 1)
```

3. Extract the `felzenszwalb` segments (which are obtained based on the color, texture, size, and shape compatibility of content within an image) from the image:

```
segments_fz = felzenszwalb(img, scale=200)
```

Note that in the `felzenszwalb` method, `scale` represents the number of clusters that can be formed within the segments of the image. The higher the value of `scale`, the greater the detail of the original image that is preserved.

4. Plot the original image and the image with segmentation:

```
subplots([img, segments_fz],
         titles=['Original Image',
                 'Image post\nfelzenszwalb segmentation'], sz=10, nc=2)
```

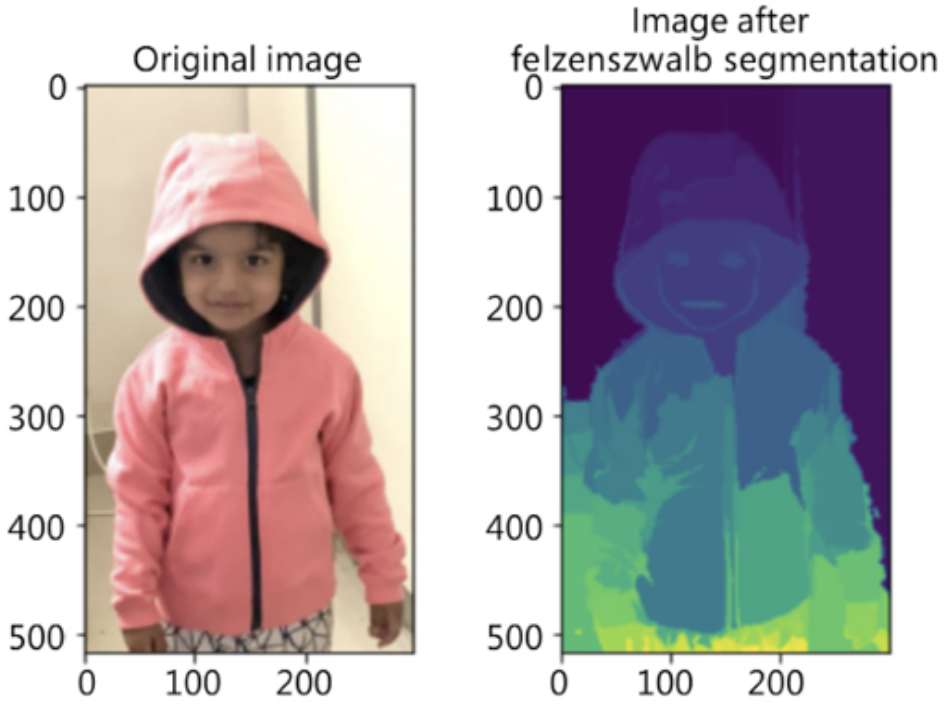


Figure 7.4: Original image and its corresponding segmentation

From the preceding output, note that the pixels belonging to the same group have similar pixel values.

Pixels that have similar values form a region proposal. This helps in object detection, as we now pass each region proposal to a network and ask it to predict whether the region proposal is a background or an object. Furthermore, if it is an object, it helps us to identify the offset to fetch the tight bounding box corresponding to the object and the class corresponding to the content within the region proposal.

Now that we understand what SelectiveSearch does, let's implement the `selectivesearch` function to fetch region proposals for the given image.

Implementing SelectiveSearch to generate region proposals

In this section, we will define the `extract_candidates` function using `selectivesearch` so that it can be leveraged in the subsequent sections on training R-CNN- and Fast R-CNN-based custom object detectors:

1. Import the relevant packages and fetch an image:

```
%pip install selectivesearch
%pip install torch_snippets
from torch_snippets import *
import selectivesearch
!wget https://www.dropbox.com/s/l98leemr7/Hemanvi.jpeg
img = read('Hemanvi.jpeg', 1)
```

2. Define the `extract_candidates` function, which fetches the region proposals from an image:
    1. Define the function that takes an image as the input parameter:

```
def extract_candidates(img):
```

    2. Fetch the candidate regions within the image using the `selective_search` method, available in the `selectivesearch` package:

```
    img_lbl, regions = selectivesearch.selective_search(img,
                                    scale=200,  min_size=100)
```

    3. Calculate the image area and initialize a list of candidates that we will use to store the candidates that pass a defined threshold:

```
    img_area = np.prod(img.shape[:2])
    candidates = []
```

    4. Fetch only those candidates (regions) that are over 5% of the total image area and less than or equal to 100% of the image area, and then return them:

```
    for r in regions:
        if r['rect'] in candidates: continue
        if r['size'] < (0.05*img_area): continue
        if r['size'] > (1*img_area): continue
        x, y, w, h = r['rect']
```

```
            candidates.append(list(r['rect']))
        return candidates
```

3. Extract the candidates and plot them on top of an image:

```
candidates = extract_candidates(img)
show(img, bbs=candidates)
```

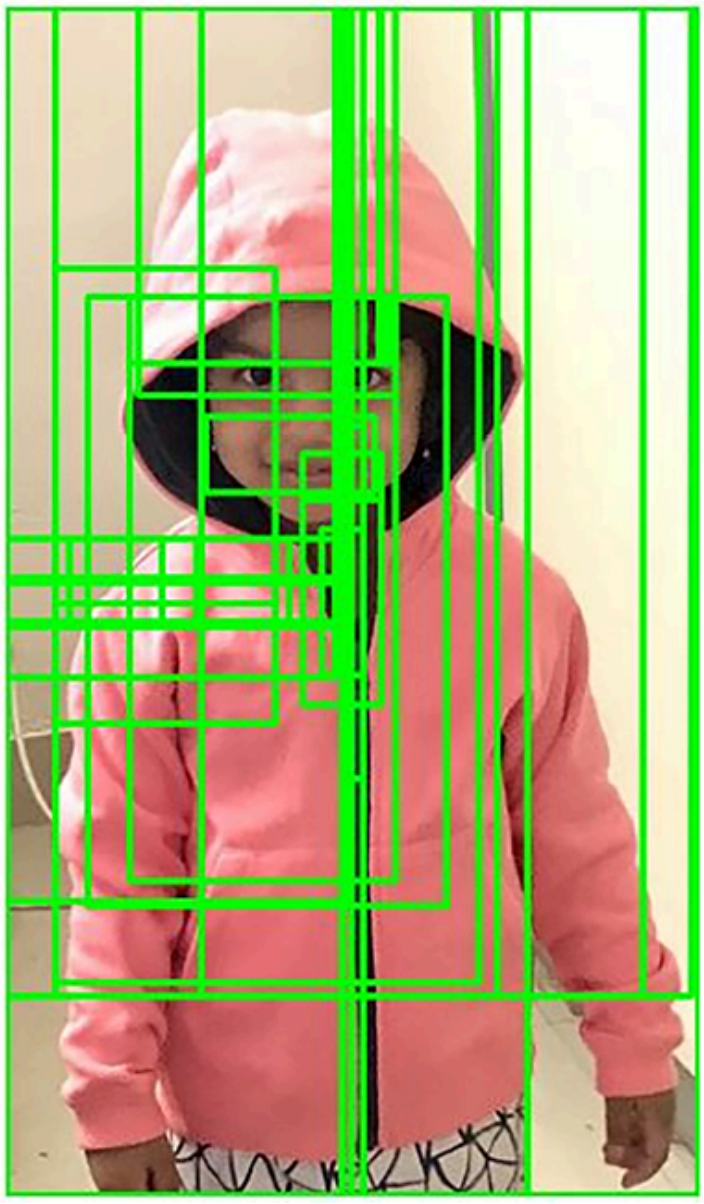


Figure 7.5: Region proposals within an image

The grids in the preceding figure represent the candidate regions (region proposals) coming from the `selective_search` method.

Now that we understand region proposal generation, one question remains unanswered. How do we leverage region proposals for object detection and localization?

A region proposal that has a high intersection with the location (ground truth) of an object in the image of interest is labeled as the one that contains the object, and a region proposal with a low intersection is labeled as the background. In the next section, we will learn how to calculate the intersection of a region proposal candidate with a ground-truth bounding box in our journey to understanding the various techniques that form the backbone of building an object detection model.

Understanding IoU

Imagine a scenario where we came up with a prediction of a bounding box for an object. How do we measure the accuracy of our prediction? The concept IoU comes in handy in such a scenario.

The word *intersection* within the term *intersection over union* refers to measuring how much the predicted and actual bounding boxes overlap, while *union* refers to measuring the overall space possible for overlap. IoU is the ratio of the overlapping region between the two bounding boxes over the combined region of both bounding boxes.

This can be represented in a diagram, as follows:

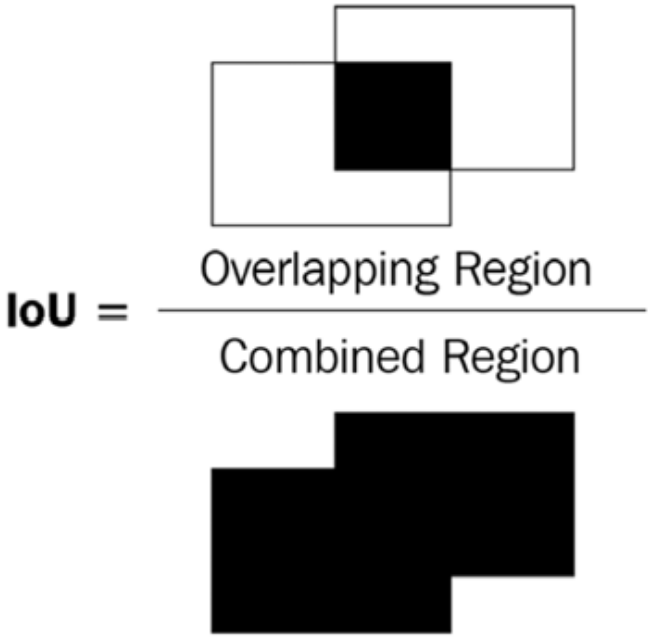


Figure 7.6: Visualizing IoU

In the preceding diagram of two bounding boxes (rectangles), let's consider the left bounding box as the ground truth and the right bounding box as the predicted location of the object. IoU as a metric is the ratio of the overlapping region over the combined region between the two bounding boxes. In the following diagram, you can observe the variation in the IoU metric as the overlap between bounding boxes varies:

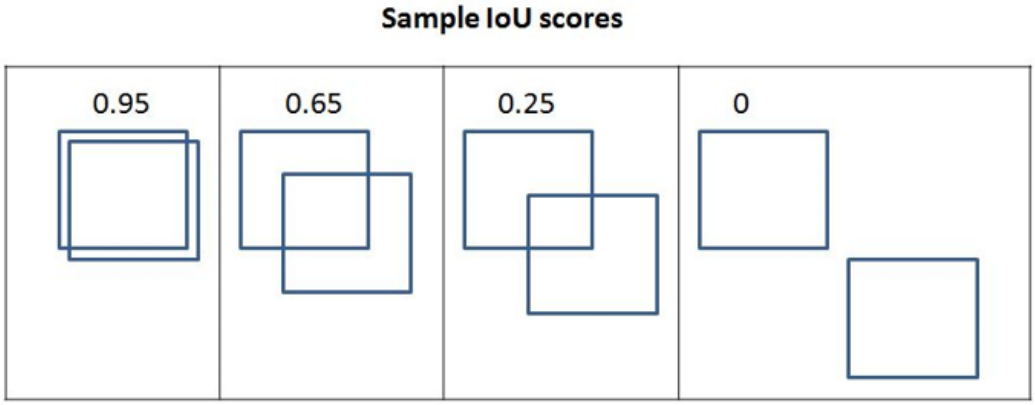


Figure 7.7: Variation of the IoU value in different scenarios

We can see that as the overlap decreases, IoU decreases, and in the final diagram, where there is no overlap, the IoU metric is 0.

Now that we understand how to measure IoU, let's implement it in code and create a function to calculate IoU as we will leverage it in the sections on training R-CNN and training Fast R-CNN.

Find the following code in the `Calculating_Intersection_Over_Union.ipynb` file in the `Chapter07` folder on GitHub at **https://bit.ly/mcvp-2e**.

Let's define a function that takes two bounding boxes as input and returns IoU as the output:

1. Specify the `get_iou` function, which takes `boxA` and `boxB` as inputs, where `boxA` and `boxB` are two different bounding boxes (you can consider `boxA` as the ground-truth bounding box and `boxB` as the region proposal):

```
def get_iou(boxA, boxB, epsilon=1e-5):
```

We define the `epsilon` parameter to address the rare scenario where the union between the two boxes is 0, resulting in a division-by-zero error. Note that in each of the bounding boxes, there will be four values corresponding to the four corners of the bounding box.

2. Calculate the coordinates of the intersection box:

```
    x1 = max(boxA[0], boxB[0])
    y1 = max(boxA[1], boxB[1])
    x2 = min(boxA[2], boxB[2])
    y2 = min(boxA[3], boxB[3])
```

Note that `x1` stores the maximum value of the left-most x value between the two bounding boxes. Similarly, `y1` stores the top-most y value, and `x2` and `y2` store the right-most x value and bottom-most y value, respectively, corresponding to the intersection part.

3. Calculate `width` and `height` corresponding to the intersection area (overlapping region):

```
    width = (x2 - x1)
    height = (y2 - y1)
```

4. Calculate the area of overlap (`area_overlap`):

```
    if (width<0) or (height <0):
        return 0.0
    area_overlap = width * height
```

Note that, in the preceding code, we specify that if the width or height corresponding to the overlapping region is less than 0, the area of intersection is 0. Otherwise, we calculate the area of overlap (intersection) similar to the way a rectangle's area is calculated – width multiplied by height.

5. Calculate the combined area corresponding to the two bounding boxes:

```
    area_a = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    area_b = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    area_combined = area_a + area_b - area_overlap
```

In the preceding code, we calculated the combined area of the two bounding boxes – `area_a` and `area_b` – and then subtracted the overlapping area while calculating `area_combined` , since `area_overlap` is counted twice – once when calculating `area_a` and then when calculating `area_b` .

6. Calculate the IoU value and return it:

```
    iou = area_overlap / (area_combined+epsilon)
    return iou
```

In the preceding code, we calculated `iou` as the ratio of the area of overlap (`area_overlap`) over the area of the combined region (`area_combined`) and returned the result.

So far, we have learned how to create the ground truth and calculate IoU, which helps prepare training data.

We will hold off on building a model until later sections, as training a model is more involved, and we would also have to learn a few more components before we train it. In the next section, we will learn about non-max suppression, which helps in narrowing down the different possible predicted bounding boxes around an object when inferring, using the trained model on a new image.

Non-max suppression

Imagine a scenario where multiple region proposals are generated and significantly overlap one another. Essentially, all the predicted bounding-box coordinates (offsets to region proposals) significantly overlap one another. For example, let's consider the following image, where multiple region proposals are generated for the person in the image:

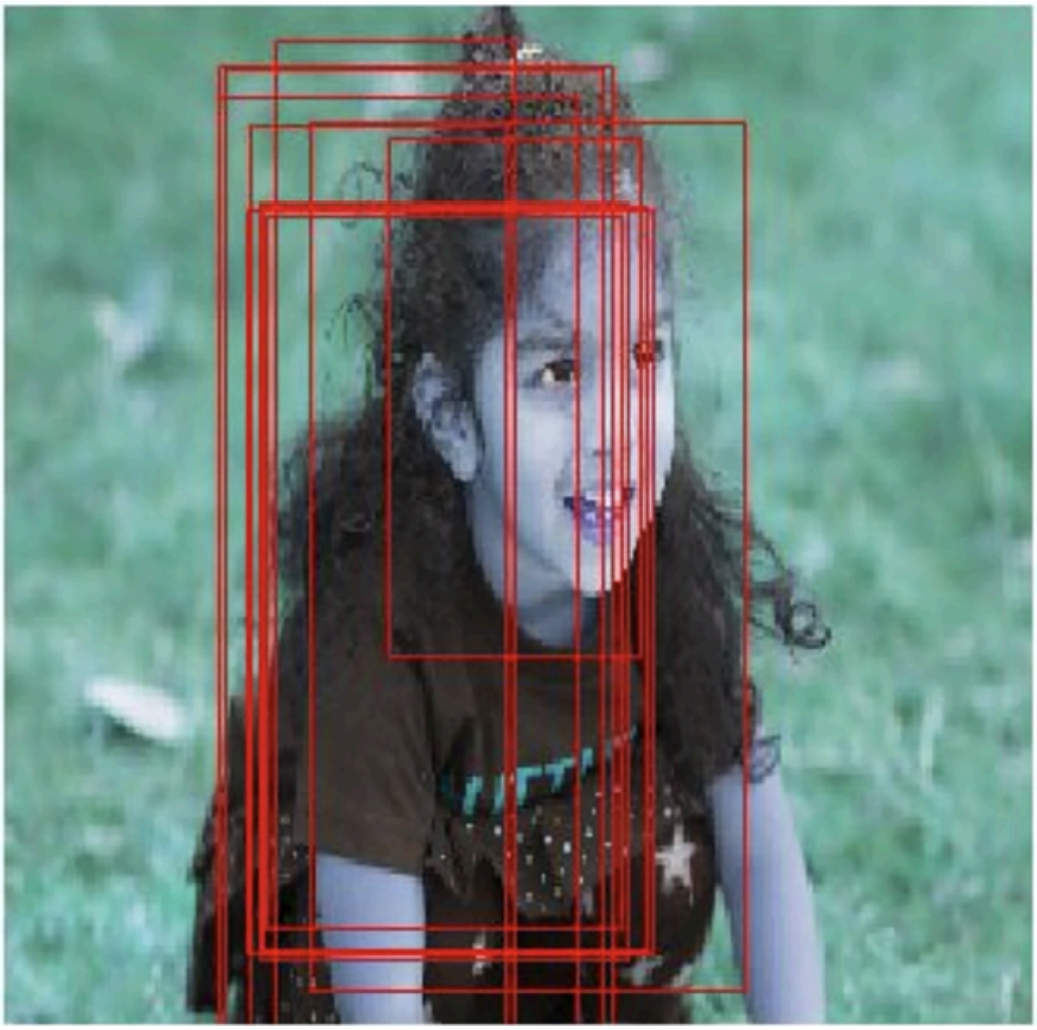


Figure 7.8: Image and the possible bounding boxes

How do we identify the box, among the many region proposals, that we will consider as the one containing an object and the boxes that we will discard? **Non-max suppression** comes in handy in such a scenario. Let's unpack that term.

Non-max refers to the boxes that don't have the highest probability of containing an object, and **suppression** refers to us discarding those boxes. In non-max suppression, we identify the bounding box that has the highest probability of containing the object and discard all the other bounding boxes that have an IoU below a certain threshold with the box showing the highest probability of containing an object.

In PyTorch, non-max suppression is performed using the `nms` function in the `torchvision.ops` module. The `nms` function takes the bounding-box coordinates, the confidence of the object in the bounding box, and the threshold of IoU across bounding boxes, identifying the bounding boxes to be retained. You will leverage the `nms` function when predicting object classes and the bounding boxes of objects in a new image in both the *Training R-CNN-based custom object detectors* and *Training Fast R-CNN-based custom object detectors* sections.

Mean average precision

So far, we have looked at getting an output that comprises a bounding box around each object within an image and the class corresponding to the object within the bounding box. Now comes the next question: how do we quantify the accuracy of the predictions coming from our model? mAP comes to the rescue in such a scenario.

Before we try to understand mAP, let's first understand precision, then average precision, and finally, mAP:

- **Precision**: Typically, we calculate precision as:

$$Precision = \frac{True\ positives}{(True\ positives + False\ positives)}$$

A true positive refers to the bounding boxes that predicted the correct class of objects and have an IoU with a ground truth that is greater than a certain threshold. A false positive refers to the bounding boxes that predicted the class incorrectly or have an overlap that is less than the defined threshold with the ground truth. Furthermore, if there are multiple bounding boxes that are identified for the same ground-truth bounding box, only one box can be a true positive, and everything else is a false positive.

- **Average precision:** Average precision is the average of precision values calculated at various IoU thresholds.
- **mAP:** mAP is the average of precision values calculated at various IoU threshold values across all the classes of objects present within a dataset.

So far, we have looked at preparing a training dataset for our model, performing non-max suppression on the model's predictions, and calculating its accuracies. In the following sections, we will learn about training a model (R-CNN-based and Fast R-CNN-based) to detect objects in new images.

Training R-CNN-based custom object detectors

R-CNN stands for **region-based convolutional neural network**. **Region-based** within R-CNN refers to the region proposals used to identify objects within an image. Note that R-CNN assists in identifying both the objects present in the image and their location within it.

In the following sections, we will learn about the working details of R-CNN before training it on our custom dataset.

Working details of R-CNN

Let's get an idea of R-CNN-based object detection at a high level using the following diagram:

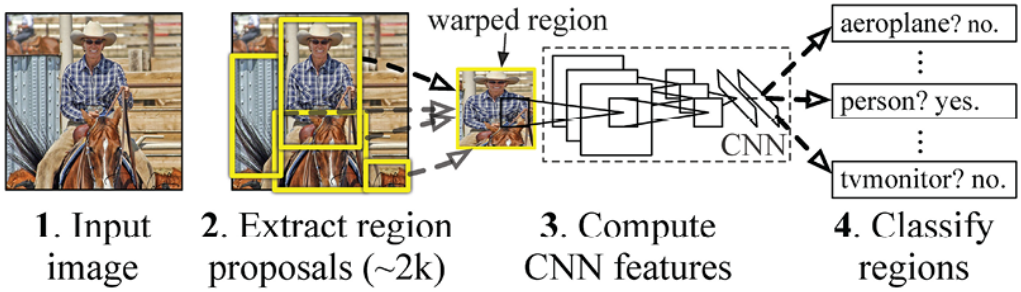


Figure 7.9: Sequence of steps for R-CNN (image source: `https://arxiv.org/pdf/1311.2524.pdf`)

We perform the following steps when leveraging the R-CNN technique for object detection:

1. Extract region proposals from an image. We need to ensure that we extract a high number of proposals to not miss out on any potential object within the image.
2. Resize (warp) all the extracted regions to get regions of the same size.
3. Pass the resized region proposals through a network. Typically, we pass the resized region proposals through a pretrained model, such as VGG16 or ResNet50, and extract the features in a fully connected layer.
4. Create data for model training, where the input is features extracted by passing the region proposals through a pretrained model. The outputs are the class corresponding to each region proposal and the offset of the region proposal from the ground truth corresponding to the image.

If a region proposal has an IoU greater than a specific threshold with the object, we create training data. In this scenario, the region is tasked with predicting both the class of the object it overlaps with and the offset of the region proposal related to the ground-truth bounding box that encompasses the object of interest. A sample image, region proposal, and ground-truth bounding box are shown as follows:

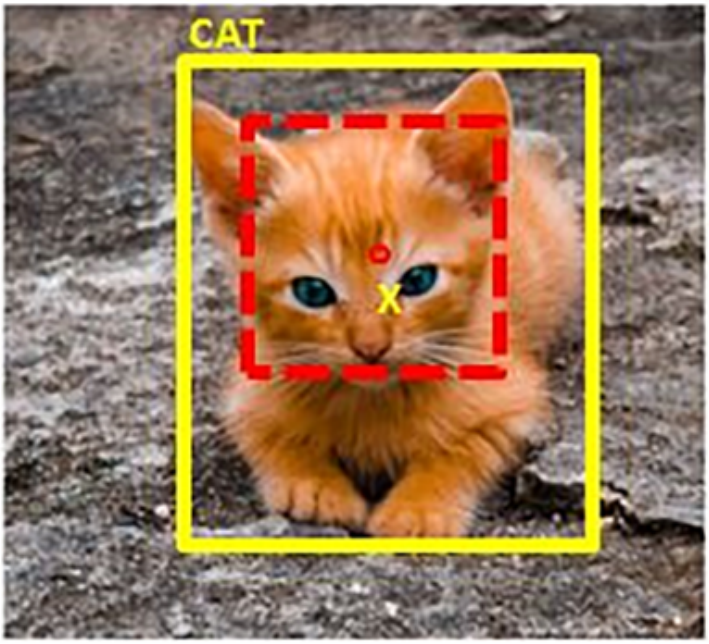


Figure 7.10: Sample image with the region proposal and ground-truth bounding box

In the preceding image, `o` (in red) represents the center of the region proposal (dotted bounding box) and `x` represents the center of the ground-truth bounding box (solid bounding box) corresponding to the `cat` class. We calculate the offset between the region proposal bounding box and the ground-truth bounding box as the difference between the center coordinates of the two bounding boxes (`dx` , `dy`), and the difference between the height and width of the bounding boxes (`dw` , `dh`).

5. Connect two output heads, one corresponding to the class of image and the other corresponding to the offset of region proposal with the ground-truth bounding box, to extract the fine bounding box on the object.

This exercise would be like the use case where we predicted gender (a categorical variable, analogous to the class of object in this case study) and age (a continuous variable, analogous to the offsets to be done on top of region proposals) based on the image of the face of a person in *Chapter 5*.

6. Train the model after writing a custom loss function that minimizes both the object classification error and the bounding-box offset error.

Note that the loss function we will minimize differs from the loss function that is optimized in the original paper (https://arxiv.org/pdf/1311.2524.pdf). We are doing this to reduce the complexity associated with building R-CNN and Fast R-CNN from scratch. Once you are familiar with how the model works and can build a model using the code in the next two sections, we highly encourage you to implement the model in original paper from scratch.

In the next section, we will learn about fetching datasets and creating data for training. In the section after that, we will learn about designing a model and training it, before predicting the class of objects present and their bounding boxes in a new image.

Implementing R-CNN for object detection on a custom dataset

So far, we have a theoretical understanding of how R-CNN works. We will now learn how to go about creating data for training. This process involves the following steps:

1. Downloading the dataset
2. Preparing the dataset
3. Defining the region proposal extraction and IoU calculation functions
4. Creating the training data
5. Creating input data for the model
6. Resizing the region proposals
7. Passing them through a pretrained model to fetch the fully connected layer values
8. Creating output data for the model
9. Labeling each region proposal with a class or background label
10. Defining the offset of the region proposal from the ground truth if the region proposal corresponds to an object and not a background
11. Defining and training the model
12. Predicting on new images

Let's get started with coding in the following sections.

Downloading the dataset

For the scenario of object detection, we will download the data from the Google Open Images v6 dataset (available at https://storage.googleapis.com/openimages/v5/test-annotations-bbox.csv). However, in code, we will work on only those images that are of a bus or truck to ensure that we can train images (as you will shortly notice the memory issues associated with using `selectivesearch`). We will expand the number of classes (more classes in addition to bus and truck) we will train on in *Chapter 10, Applications of Object Detection and Segmentation*:

Find the following code in the `Training_RCNN.ipynb` file in the `Chapter07` folder on GitHub at https://bit.ly/mcvp-2e. The code contains URLs to download data from and is moderately lengthy. We strongly recommend that you execute the notebook in GitHub to help you reproduce results and understand the steps and the various code components in the text.

Import the relevant packages to download the files that contain images and their ground truths:

```
%pip install -q --upgrade selectivesearch torch_snippets
from torch_snippets import *
import selectivesearch
from google.colab import files
files.upload() # upload kaggle.json file
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!ls ~/.kaggle
!chmod 600 /root/.kaggle/kaggle.json
!kaggle datasets download -d sixhky/open-images-bus-trucks/
!unzip -qq open-images-bus-trucks.zip
from torchvision import transforms, models, datasets
from torch_snippets import Report
from torchvision.ops import nms
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Once we execute the preceding code, we should have the images and their corresponding ground truths stored in an available CSV file.

Preparing the dataset

Now that we have downloaded the dataset, we will prepare the dataset. This involves the following steps:

1. Fetching each image and its corresponding class and bounding-box values
2. Fetching the region proposals within each image, their corresponding IoU, and the delta by which the region proposal will be corrected with respect to the ground truth
3. Assigning numeric labels for each class (where we have an additional background class, besides the bus and truck classes, where IoU with the ground-truth bounding box is below a threshold)
4. Resizing each region proposal to a common size in order to pass them to a network

By the end of this exercise, we will have resized the crops of region proposals, assigned the ground-truth class to each region proposal, and calculated the offset of the region proposal in relation to the ground-truth bounding box. We will continue coding from where we left off in the preceding section:

1. Specify the location of images, and read the ground truths present in the CSV file that we downloaded:

```
IMAGE_ROOT = 'images/images'
DF_RAW = pd.read_csv('df.csv')
print(DF_RAW.head())
```

	ImageID	Source	LabelName	Confidence	XMin	XMax	YMin	YMax	IsOccluded	IsTruncated	IsGroupOf	IsDepiction	IsInside
20	002f8241bd829022	xclick	Bus	1	0.257812	0.515625	0.485417	0.891667	1	0	0	0	0
21	002f8241bd829022	xclick	Bus	1	0.535937	0.907813	0.347917	0.997917	1	1	0	0	0
191	013b99371484d3d5	xclick	Bus	1	0.154688	0.920312	0.102083	0.872917	0	0	0	0	0
322	01f8886b50a031a1	xclick	Truck	1	0.012821	0.987179	0.000000	0.969512	0	0	0	0	0
405	02717d30304f4849	xclick	Bus	1	0.106250	0.926562	0.266667	0.635417	0	0	0	0	0

Figure 7.11: Sample data

Note that `XMin`, `XMax`, `YMin`, and `YMax` correspond to the ground truth of the bounding box of the image. Furthermore, `LabelName` provides the class of image.

2. Define a class that returns the image and its corresponding class and ground truth along with the file path of the image:
    1. Pass the dataframe (`df`) and the path to the folder containing images (`image_folder`) as input to the `__init__` method, and fetch the unique `ImageID` values present in the data frame (`self.unique_images`). We do this because an image can contain multiple objects, and so multiple rows can correspond to the same `ImageID` value:

```
class OpenImages(Dataset):
    def __init__(self, df, image_folder=IMAGE_ROOT):
        self.root = image_folder
        self.df = df
        self.unique_images = df['ImageID'].unique()
    def __len__(self): return len(self.unique_images)
```

    2. Define the `__getitem__` method, where we fetch the image (`image_id`) corresponding to an index (`ix`), `fetch` its bounding-box coordinates (`boxes`), and classes and return the image, bounding box, class, and image path:

```
    def __getitem__(self, ix):
        image_id = self.unique_images[ix]
        image_path = f'{self.root}/{image_id}.jpg'
        # Convert BGR to RGB
        image = cv2.imread(image_path, 1)[...,::-1]
        h, w, _ = image.shape
        df = self.df.copy()
        df = df[df['ImageID'] == image_id]
        boxes = df['XMin,YMin,XMax,YMax'.split(',')].values
        boxes = (boxes*np.array([w,h,w,h])).astype(np.uint16).tolist()
        classes = df['LabelName'].values.tolist()
        return image, boxes, classes, image_path
```

3. Inspect a sample image and its corresponding class and bounding-box ground truth:

```
ds = OpenImages(df=DF_RAW)
im, bbs, clss, _ = ds[9]
show(im, bbs=bbs, texts=clss, sz=10)
```

Figure 7.12: Sample image with the ground-truth bounding box and class of object

4. Define the `extract_iou` and `extract_candidates` functions:

```
def extract_candidates(img):
    img_lbl,regions = selectivesearch.selective_search(img,
                                    scale=200, min_size=100)
    img_area = np.prod(img.shape[:2])
    candidates = []
    for r in regions:
        if r['rect'] in candidates: continue
        if r['size'] < (0.05*img_area): continue
        if r['size'] > (1*img_area): continue
        x, y, w, h = r['rect']
        candidates.append(list(r['rect']))
```

```
    return candidates
def extract_iou(boxA, boxB, epsilon=1e-5):
    x1 = max(boxA[0], boxB[0])
    y1 = max(boxA[1], boxB[1])
    x2 = min(boxA[2], boxB[2])
    y2 = min(boxA[3], boxB[3])
    width = (x2 - x1)
    height = (y2 - y1)
    if (width<0) or (height <0):
        return 0.0
    area_overlap = width * height
    area_a = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    area_b = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    area_combined = area_a + area_b - area_overlap
    iou = area_overlap / (area_combined+epsilon)
    return iou
```

With that, we have now defined all the functions necessary to prepare data and initialize data loaders. Next, we will fetch region proposals (input regions to the model) and the ground truth of the bounding box offset, along with the class of object (expected output).

Fetching region proposals and the ground truth of offset

In this section, we will learn to create the input and output values corresponding to our model. The input constitutes the candidates that are extracted using the `selectivesearch` method, and the output constitutes the class corresponding to candidates and the offset of the candidate with respect to the bounding box it overlaps the most with if the candidate contains an object.

We will continue coding from where we ended in the preceding section:

1. Initialize empty lists to store file paths (`FPATHS`), ground truth bounding boxes (`GTBBS`), classes (`CLSS`) of objects, the delta offset of a bounding box with region proposals (`DELTAS`), region proposal locations (`ROIS`), and the IoU of region proposals with ground truths (`IOUS`):

```
FPATHS, GTBBS, CLSS, DELTAS, ROIS, IOUS = [],[],[],[],[],[]
```

2. Loop through the dataset and populate the lists initialized in *step 1*:
    1. For this exercise, we can use all the data points for training or illustrate with just the first 500 data points. You can choose between either of the two, which dictates the training time and training accuracy (the greater the data points, the greater the training time and accuracy):

```
N = 500
for ix, (im, bbs, labels, fpath) in enumerate(ds):
    if(ix==N):
        break
```

In the preceding code, we specify that we will work on 500 images.

    2. Extract candidates from each image (`im`) in absolute pixel values (note that `XMin` , `Xmax` , `YMin` , and `YMax` are available as a proportion of the shape of images in the downloaded data frame) using the `extract_candidates` function and convert the extracted regions coordinates from an (x,y,w,h) system to an (x,y,x+w,y+h) system:

```
    H, W, _ = im.shape
    candidates = extract_candidates(im)
    candidates = np.array([(x,y,x+w,y+h) for x,y,w,h in candidates])
```

    3. Initialize `ious` , `rois` , `deltas` , and `clss` as lists that store `iou` for each candidate, region proposal location, bounding box offset, and class corresponding to every candidate for each image. We will go through all the proposals from SelectiveSearch and store those with a high IOU as bus/truck proposals (whichever is the class in labels) and the rest as background proposals:

```
    ious, rois, clss, deltas = [], [], [], []
```

    4. Store the IoU of all candidates with respect to all ground truths for an image where `bbs` is the ground truth bounding box of different

objects present in the image, and `candidates` consists of the region proposal candidates obtained in the previous step:

```
    ious = np.array([[extract_iou(candidate, _bb_) for \
                candidate in candidates] for _bb_ in bbs]).T
```

5. Loop through each candidate and store the XMin (`cx`), YMin (`cy`), XMax (`cX`), and YMax (`cY`) values of a candidate:

```
    for jx, candidate in enumerate(candidates):
        cx,cy,cX,cY = candidate
```

6. Extract the IoU corresponding to the candidate with respect to all the ground truth bounding boxes that were already calculated when fetching the list of lists of IoUs:

```
        candidate_ious = ious[jx]
```

7. Find the index of a candidate (`best_iou_at`) that has the highest IoU and the corresponding ground truth (`best_bb`):

```
        best_iou_at = np.argmax(candidate_ious)
        best_iou = candidate_ious[best_iou_at]
        best_bb = _x,_y,_X,_Y = bbs[best_iou_at]
```

8. If IoU (`best_iou`) is greater than a threshold (0.3), we assign the label of the class corresponding to the candidate, and the background otherwise:

```
        if best_iou > 0.3: clss.append(labels[best_iou_at])
        else : clss.append('background')
```

9. Fetch the offsets needed (`delta`) to transform the current proposal into the candidate that is the best region proposal (which is the ground truth bounding box) – `best_bb` – in other words, how much the left, right, top, and bottom margins of the current pro-

posal should be adjusted so that it aligns exactly with `best_bb` from the ground truth:

```
        delta = np.array([_x-cx, _y-cy, _X-cX, _Y-cY]) / np.array([W,H,W,H]
        deltas.append(delta)
        rois.append(candidate / np.array([W,H,W,H]))
```

10. Append the file paths, IoU, RoI, class delta, and ground truth bounding boxes:

```
    FPATHS.append(fpath)
    IOUS.append(ious)
    ROIS.append(rois)
    CLSS.append(clss)
    DELTAS.append(deltas)
    GTBBS.append(bbs)
```

11. Fetch the image path names and store all the information obtained, `FPATHS`, `IOUS`, `ROIS`, `CLSS`, `DELTAS`, and `GTBBS`, in a list of lists:

```
FPATHS = [f'{IMAGE_ROOT}/{stem(f)}.jpg' for f in FPATHS]
FPATHS, GTBBS, CLSS, DELTAS, ROIS = [item for item in \
                                     [FPATHS, GTBBS, \
                                      CLSS, DELTAS, ROIS]]
```

Note that, so far, classes are available as the name of the class. Now, we will convert them into their corresponding indices so that a background class has a class index of 0, a bus class has a class index of 1, and a truck class has a class index of 2.

3. Assign indices to each class:

```
targets = pd.DataFrame(flatten(CLSS), columns=['label'])
label2target = {l:t for t,l in enumerate(targets['label'].unique())}
target2label = {t:l for l,t in label2target.items()}
background_class = label2target['background']
```

With that, we have assigned a class to each region proposal and also created the other ground truth of the bounding box offset. Next, we will fetch the dataset and the data loaders corresponding to the information obtained (`FPATHS`, `IOUS`, `ROIS`, `CLSS`, `DELTAS`, and `GTBBS`).

Creating the training data

So far, we have fetched data, fetched region proposals across all images, prepared the ground truths of the class of object present within each region proposal, and the offset corresponding to each region proposal that has a high overlap (IoU) with the object in the corresponding image.

In this section, we will prepare a dataset class based on the ground truth of region proposals that were obtained at the end of *step 3 in the previous section*, creating data loaders from it. Then, we will normalize each region proposal by resizing them to the same shape and scaling them. We will continue coding from where we left off in the preceding section:

1. Define the function to normalize an image before passing through a pretrained model like VGG16. The standard normalization practice for VGG16 is as follows:

```
normalize= transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                 std=[0.229, 0.224, 0.225])
```

2. Define a function (`preprocess_image`) to preprocess the image (`img`), where we switch channels, normalize the image, and register it with the device:

```
def preprocess_image(img):
    img = torch.tensor(img).permute(2,0,1)
    img = normalize(img)
    return img.to(device).float()
```

   Define the function to `decode` prediction:

```
def decode(_y):
    _, preds = _y.max(-1)
```

```
        return preds
```

3. Define the dataset (`RCNNDataset`) using the preprocessed region proposals, along with the ground truths obtained in the previous step (*step 2 of the previous section*):

```
class RCNNDataset(Dataset):
    def __init__(self, fpaths, rois, labels, deltas, gtbbs):
        self.fpaths = fpaths
        self.gtbbs = gtbbs
        self.rois = rois
        self.labels = labels
        self.deltas = deltas
    def __len__(self): return len(self.fpaths)
```

1. Fetch the crops as per the region proposals, along with the other ground truths related to the class and bounding box offset:

```
    def __getitem__(self, ix):
        fpath = str(self.fpaths[ix])
        image = cv2.imread(fpath, 1)[...,::-1]
        H, W, _ = image.shape
        sh = np.array([W,H,W,H])
        gtbbs = self.gtbbs[ix]
        rois = self.rois[ix]
        bbs = (np.array(rois)*sh).astype(np.uint16)
        labels = self.labels[ix]
        deltas = self.deltas[ix]
        crops = [image[y:Y,x:X] for (x,y,X,Y) in bbs]
        return image,crops,bbs,labels,deltas,gtbbs,fpath
```

2. Define `collate_fn`, which performs the resizing and normalizing (`preprocess_image`) of an image of a crop:

```
    def collate_fn(self, batch):
        input, rois, rixs, labels, deltas =[],[],[],[],[]
        for ix in range(len(batch)):
            image, crops, image_bbs, image_labels, \
                image_deltas, image_gt_bbs, \
                image_fpath = batch[ix]
```

```
            crops = [cv2.resize(crop, (224,224)) for crop in crops]
            crops = [preprocess_image(crop/255.)[None] for crop in crops]
            input.extend(crops)
            labels.extend([label2target[c] for c in image_labels])
            deltas.extend(image_deltas)
        input = torch.cat(input).to(device)
        labels = torch.Tensor(labels).long().to(device)
        deltas = torch.Tensor(deltas).float().to(device)
        return input, labels, deltas
```

4. Create the training and validation datasets and the data loaders:

```
n_train = 9*len(FPATHS)//10
train_ds = RCNNDataset(FPATHS[:n_train],ROIS[:n_train],
                       CLSS[:n_train], DELTAS[:n_train],
                       GTBBS[:n_train])
test_ds = RCNNDataset(FPATHS[n_train:], ROIS[n_train:],
                      CLSS[n_train:], DELTAS[n_train:],
                      GTBBS[n_train:])
from torch.utils.data import TensorDataset, DataLoader
train_loader = DataLoader(train_ds, batch_size=2,
                          collate_fn=train_ds.collate_fn,
                          drop_last=True)
test_loader = DataLoader(test_ds, batch_size=2,
                         collate_fn=test_ds.collate_fn,
                         drop_last=True)
```

So far, we have learned about preparing data for training. Next, we will learn about defining and training a model that predicts the class and offset to be made to a region proposal, to fit a tight bounding box around objects in an image.

R-CNN network architecture

Now that we have prepared the data, in this section, we will learn about building a model that can predict both the class of a region proposal and the offset corresponding to it, in order to draw a tight bounding box around the object in an image. The strategy we adopt is as follows:

1. Define a VGG backbone.

2. Fetch the features after passing the normalized crop through a pre-trained model.
3. Attach a linear layer with sigmoid activation to the VGG backbone to predict the class corresponding to the region proposal.
4. Attach an additional linear layer to predict the four bounding-box offsets.
5. Define the loss calculations for each of the two outputs (one to predict the class and the other to predict the four bounding-box offsets).
6. Train the model that predicts both the class of region proposal and the four bounding-box offsets.

Execute the following code. We will continue coding from where we ended in the preceding section:

1. Define a VGG backbone:

```
vgg_backbone = models.vgg16(pretrained=True)
vgg_backbone.classifier = nn.Sequential()
for param in vgg_backbone.parameters():
    param.requires_grad = False
vgg_backbone.eval().to(device)
```

2. Define the `RCNN` network module:
    1. Define the class:

```
class RCNN(nn.Module):
    def __init__(self):
        super().__init__()
```

    2. Define the backbone (`self.backbone`) and how we calculate the class score (`self.cls_score`) and the bounding-box offset values (`self.bbox`):

```
        feature_dim = 25088
        self.backbone = vgg_backbone
        self.cls_score = nn.Linear(feature_dim, len(label2target))
        self.bbox = nn.Sequential(
                        nn.Linear(feature_dim, 512),
```

```
                nn.ReLU(),
                nn.Linear(512, 4),
                nn.Tanh(),
            )
```

3. Define the loss functions corresponding to class prediction (`self.cel`) and bounding-box offset regression (`self.sl1`):

```
        self.cel = nn.CrossEntropyLoss()
        self.sl1 = nn.L1Loss()
```

4. Define the feed-forward method where we pass the image through a VGG backbone (`self.backbone`) to fetch features (`feat`), which are further passed through the methods corresponding to classification and bounding-box regression to fetch the probabilities across classes (`cls_score`) and the bounding-box offsets (`bbox`):

```
    def forward(self, input):
        feat = self.backbone(input)
        cls_score = self.cls_score(feat)
        bbox = self.bbox(feat)
        return cls_score, bbox
```

5. Define the function to calculate loss (`calc_loss`), where we return the sum of detection and regression loss. Note that we do not calculate regression loss corresponding to offsets if the actual class is the background class:

```
    def calc_loss(self, probs, _deltas, labels, deltas):
        detection_loss = self.cel(probs, labels)
        ixs, = torch.where(labels != 0)
        _deltas = _deltas[ixs]
        deltas = deltas[ixs]
        self.lmb = 10.0
        if len(ixs) > 0:
            regression_loss = self.sl1(_deltas, deltas)
            return detection_loss + self.lmb *\
                regression_loss, detection_loss.detach(),
                regression_loss.detach()
```

```
        else:
            regression_loss = 0
            return detection_loss + self.lmb regression_loss, \
                   detection_loss.detach(), regression_loss
```

3. With the model class in place, we now define the functions to train on a batch of data and predict on validation data:
    1. Define the `train_batch` function:

```
def train_batch(inputs, model, optimizer, criterion):
    input, clss, deltas = inputs
    model.train()
    optimizer.zero_grad()
    _clss, _deltas = model(input)
    loss, loc_loss, regr_loss = criterion(_clss, _deltas, clss, deltas)
    accs = clss == decode(_clss)
    loss.backward()
    optimizer.step()
    return loss.detach(), loc_loss, regr_loss, accs.cpu().numpy()
```

    2. Define the `validate_batch` function:

```
@torch.no_grad()
def validate_batch(inputs, model, criterion):
    input, clss, deltas = inputs
    with torch.no_grad():
        model.eval()
        _clss,_deltas = model(input)
        loss,loc_loss,regr_loss = criterion(_clss, _deltas, clss, deltas)
        _, _clss = _clss.max(-1)
        accs = clss == _clss
    return _clss,_deltas,loss.detach(), \
                 loc_loss, regr_loss, accs.cpu().numpy()
```

4. Now, let's create an object of the model, fetch the loss criterion, and then define the optimizer and the number of epochs:

```
rcnn = RCNN().to(device)
criterion = rcnn.calc_loss
optimizer = optim.SGD(rcnn.parameters(), lr=1e-3)
```

```
n_epochs = 5
log = Report(n_epochs)
```

We now train the model over increasing epochs:

```
for epoch in range(n_epochs):
    _n = len(train_loader)
    for ix, inputs in enumerate(train_loader):
        loss, loc_loss,regr_loss,accs = train_batch(inputs, rcnn,
                                        optimizer, criterion)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, trn_loss=loss.item(),
                   trn_loc_loss=loc_loss,
                   trn_regr_loss=regr_loss,
                   trn_acc=accs.mean(), end='\r')

    _n = len(test_loader)
    for ix,inputs in enumerate(test_loader):
        _clss, _deltas, loss, loc_loss, regr_loss, \
        accs = validate_batch(inputs, rcnn, criterion)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, val_loss=loss.item(),
                   val_loc_loss=loc_loss,
                   val_regr_loss=regr_loss,
                   val_acc=accs.mean(), end='\r')
# Plotting training and validation metrics
log.plot_epochs('trn_loss,val_loss'.split(','))
```

The plot of overall loss across training and validation data is as follows:

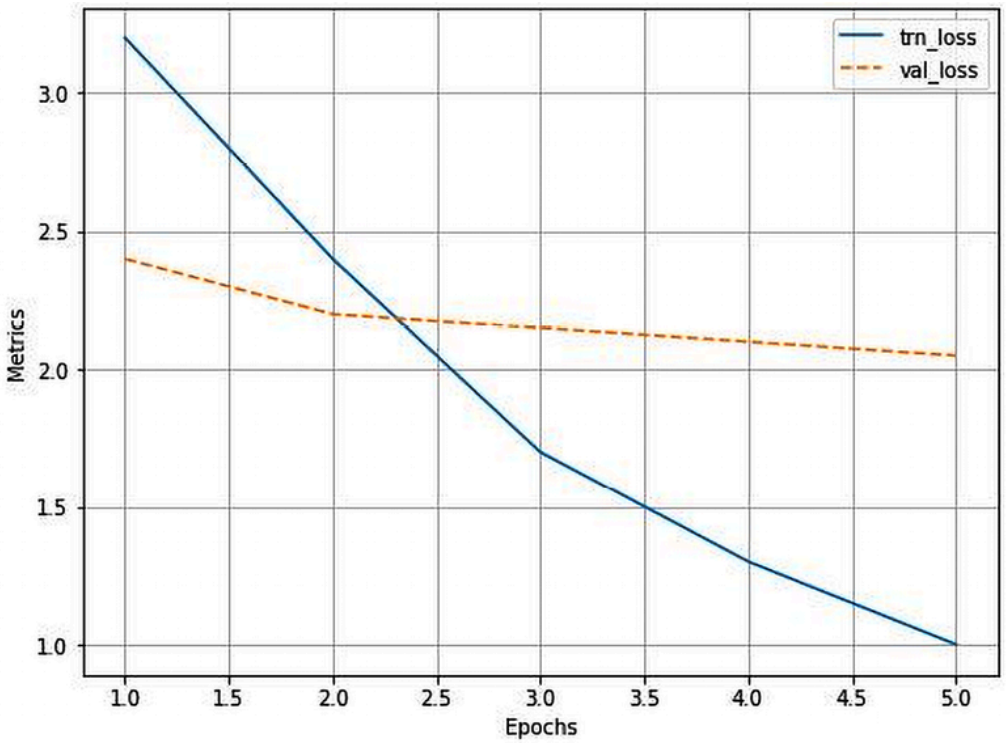


Figure 7.13: Training and validation loss over increasing epochs

Now that we have trained a model, we will use it to predict on a new image in the next section.

Predicting on a new image

Let's leverage the model trained so far to predict and draw bounding boxes around objects and the corresponding class of an object, within the predicted bounding box on new images. The strategy we adopt is as follows:

1. Extract region proposals from the new image.
2. Resize and normalize each crop.
3. Feed-forward the processed crops to predict the class and the offsets.
4. Perform non-max suppression to fetch only those boxes that have the highest confidence of containing an object.

We execute the preceding strategy through a function that takes an image as input and a ground-truth bounding box (this is used only so that we compare the ground truth and the predicted bounding box).

We will continue coding from where we left off in the preceding section:

1. Define the `test_predictions` function to predict on a new image:
    1. The function takes `filename` as input, reads the image, and extracts the candidates:

```
def test_predictions(filename, show_output=True):
    img = np.array(cv2.imread(filename, 1)[...,::-1])
    candidates = extract_candidates(img)
    candidates = [(x,y,x+w,y+h) for x,y,w,h in candidates]
```

    2. Loop through the candidates to resize and preprocess the image:

```
    input = []
    for candidate in candidates:
        x,y,X,Y = candidate
        crop = cv2.resize(img[y:Y,x:X], (224,224))
        input.append(preprocess_image(crop/255.)[None])
    input = torch.cat(input).to(device)
```

    3. Predict the class and offset:

```
    with torch.no_grad():
        rcnn.eval()
        probs, deltas = rcnn(input)
        probs = torch.nn.functional.softmax(probs, -1)
        confs, clss = torch.max(probs, -1)
```

    4. Extract the candidates that do not belong to the background class and sum up the candidates' bounding box with the predicted bounding-box offset values:

```
    candidates = np.array(candidates)
    confs,clss,probs,deltas=[tensor.detach().cpu().numpy() \
                             for tensor in [confs, clss, probs, deltas]]
    ixs = clss!=background_class
    confs,clss,probs,deltas,candidates = [tensor[ixs] for \
```

```
            tensor in [confs,clss, probs, deltas,candidates]]
    bbs = (candidates + deltas).astype(np.uint16)
```

5. Use non-max suppression (`nms`) to eliminate near-duplicate bounding boxes (pairs of boxes that have an IoU greater than 0.05 are considered duplicates in this case). Among the duplicated boxes, we pick the box with the highest confidence and discard the rest:

```
    ixs = nms(torch.tensor(bbs.astype(np.float32)),
                                torch.tensor(confs), 0.05)
    confs,clss,probs,deltas,candidates,bbs=[tensor[ixs] \
              for tensor in [confs, clss, probs, deltas, candidates, bbs]]
    if len(ixs) == 1:
        confs, clss, probs, deltas, candidates, bbs = \
                [tensor[None] for tensor in [confs, clss,
                          probs, deltas, candidates, bbs]]
```

6. Fetch the bounding box with the highest confidence:

```
    if len(confs) == 0 and not show_output:
        return (0,0,224,224), 'background', 0
    if len(confs) > 0:
        best_pred = np.argmax(confs)
        best_conf = np.max(confs)
        best_bb = bbs[best_pred]
        x,y,X,Y = best_bb
```

7. Plot the image along with the predicted bounding box:

```
    _, ax = plt.subplots(1, 2, figsize=(20,10))
    show(img, ax=ax[0])
    ax[0].grid(False)
    ax[0].set_title('Original image')
    if len(confs) == 0:
        ax[1].imshow(img)
        ax[1].set_title('No objects')
        plt.show()
        return
    ax[1].set_title(target2label[clss[best_pred]])
    show(img, bbs=bbs.tolist(),
```

```
            texts=[target2label[c] for c in clss.tolist()],
            ax=ax[1], title='predicted bounding box and class')
        plt.show()
        return (x,y,X,Y),target2label[clss[best_pred]],best_conf
```

2. Execute the preceding function on a new image:

```
image, crops, bbs, labels, deltas, gtbbs, fpath = test_ds[7]
test_predictions(fpath)
```

The preceding code generates the following images:

Figure 7.14: Original image and the predicted bounding box and class

From the preceding figure, we can see that the prediction of the class of an image is accurate and the bounding-box prediction is decent, too. Note that it took ~1.5 seconds to generate a prediction for the preceding image.

All of this time is spent generating region proposals, resizing each region proposal, passing them through a VGG backbone, and generating predictions using the defined model. The most time spent is in passing each proposal through a VGG backbone. In the next section, we will learn about getting around this *passing each proposal to VGG* problem by using the Fast R-CNN architecture-based model.

Training Fast R-CNN-based custom object detectors

One of the major drawbacks of R-CNN is that it takes considerable time to generate predictions. This is because generating region proposals for each image, resizing the crops of regions, and extracting features corresponding to **each crop** (region proposal) create a bottleneck.

Fast R-CNN gets around this problem by passing the **entire image** through the pretrained model to extract features, and then it fetches the region of features that correspond to the region proposals (which are obtained from `selectivesearch`) of the original image. In the following sections, we will learn about the working details of Fast R-CNN before training it on our custom dataset.

Working details of Fast R-CNN

Let's understand Fast R-CNN through the following diagram:

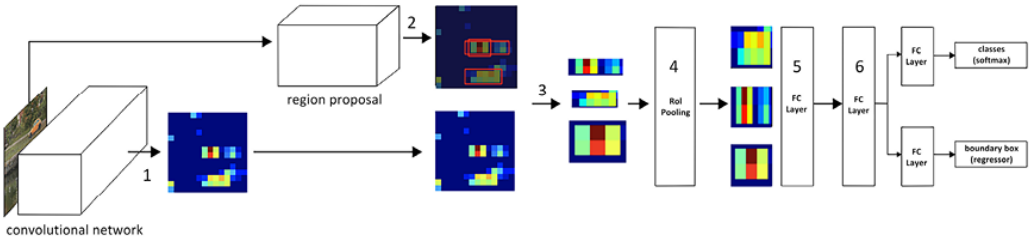


Figure 7.15: Working details of Fast R-CNN

Let's understand the preceding diagram through the following steps:

1. Pass the image through a pretrained model to extract features prior to the flattening layer; let's call them output feature maps.
2. Extract region proposals corresponding to the image.
3. Extract the feature map area corresponding to the region proposals (note that when an image is passed through a VGG16 architecture, the image is downscaled by 32 at the output, as five pooling operations are performed. Thus, if a region exists with a bounding box of (32,64,160,128) in the original image, the feature map corresponding to the bounding box of (1,2,5,4) would correspond to the exact same region).

4. Pass the feature maps corresponding to region proposals through the **region of interest** (**RoI**) pooling layer one at a time so that all feature maps of region proposals have a similar shape. This is a replacement for the warping that was executed in the R-CNN technique.
5. Pass the RoI pooling layer output value through a fully connected layer.
6. Train the model to predict the class and offsets corresponding to each region proposal.

Note that the big difference between R-CNN and Fast R-CNN is that, in R-CNN, we pass the crops (resized region proposals) through the pretrained model one at a time, while in Fast R-CNN, we crop the feature map (which is obtained by passing the whole image through a pretrained model) corresponding to each region proposal, thereby avoiding the need to pass each resized region proposal through the pretrained model.

Now armed with an understanding of how Fast R-CNN works, in the next section, we will build a model using the same dataset that we leveraged in the R-CNN section.

Implementing Fast R-CNN for object detection on a custom dataset

In this section, we will work toward training our custom object detector using Fast R-CNN. To remain succinct, we provide only the additional or changed code in this section (you should run all the code until *step 2* in the *Creating the training data* subsection in the previous section about R-CNN):

Here, we only provide the additional code to train Fast R-CNN. Find the full code in the `Training_Fast_R_CNN.ipynb` file in the `Chapter07` folder on GitHub at **https://bit.ly/mcvp-2e**.

1. Create an `FRCNNDataset` class that returns images, labels, ground truths, region proposals, and the delta corresponding to each region proposal:

```
class FRCNNDataset(Dataset):
    def __init__(self, fpaths, rois, labels, deltas, gtbbs):
        self.fpaths = fpaths
        self.gtbbs = gtbbs
        self.rois = rois
        self.labels = labels
        self.deltas = deltas
    def __len__(self): return len(self.fpaths)
    def __getitem__(self, ix):
        fpath = str(self.fpaths[ix])
        image = cv2.imread(fpath, 1)[...,::-1]
        gtbbs = self.gtbbs[ix]
        rois = self.rois[ix]
        labels = self.labels[ix]
        deltas = self.deltas[ix]
        assert len(rois) == len(labels) == len(deltas), \
            f'{len(rois)}, {len(labels)}, {len(deltas)}'
        return image, rois, labels, deltas, gtbbs, fpath
    def collate_fn(self, batch):
        input, rois, rixs, labels, deltas = [],[],[],[],[]
        for ix in range(len(batch)):
            image, image_rois, image_labels, image_deltas, \
                image_gt_bbs, image_fpath = batch[ix]
            image = cv2.resize(image, (224,224))
            input.append(preprocess_image(image/255.)[None])
            rois.extend(image_rois)
            rixs.extend([ix]*len(image_rois))
            labels.extend([label2target[c] for c in image_labels])
            deltas.extend(image_deltas)
        input = torch.cat(input).to(device)
        rois = torch.Tensor(rois).float().to(device)
        rixs = torch.Tensor(rixs).float().to(device)
        labels = torch.Tensor(labels).long().to(device)
        deltas = torch.Tensor(deltas).float().to(device)
        return input, rois, rixs, labels, deltas
```

Note that the preceding code is very similar to what we have learned in the R-CNN section, with the only change being that we are returning

more information (`rois` and `rixs`).

The `rois` matrix holds information regarding which RoI belongs to which image in the batch. Note that `input` contains multiple images, whereas `rois` is a single list of boxes. We wouldn't know how many RoIs belong to the first image, how many belong to the second image, and so on. This is where `rixs` comes into the picture. It is a list of indexes. Each integer in the list associates the corresponding bounding box with the appropriate image; for example, if `rixs` is `[0,0,0,1,1,2,3,3,3]`, then we know that the first three bounding boxes belong to the first image in the batch, and the next two belong to the second image in the batch.

2. Create training and test datasets:

```
n_train = 9*len(FPATHS)//10
train_ds = FRCNNDataset(FPATHS[:n_train],ROIS[:n_train],
                        CLSS[:n_train], DELTAS[:n_train],
                        GTBBS[:n_train])
test_ds = FRCNNDataset(FPATHS[n_train:], ROIS[n_train:],
                       CLSS[n_train:], DELTAS[n_train:],
                       GTBBS[n_train:])
from torch.utils.data import TensorDataset, DataLoader
train_loader = DataLoader(train_ds, batch_size=2,
                          collate_fn=train_ds.collate_fn,
                          drop_last=True)
test_loader = DataLoader(test_ds, batch_size=2,
                         collate_fn=test_ds.collate_fn,
                         drop_last=True)
```

3. Define a model to train on the dataset:
    1. First, import the `RoIPool` method present in the `torchvision.ops` class:

```
from torchvision.ops import RoIPool
```

    2. Define the `FRCNN` network module:

```
class FRCNN(nn.Module):
    def __init__(self):
```

```
        super().__init__()
```

3. Load the pretrained model and freeze the parameters:

```
        rawnet= torchvision.models.vgg16_bn(pretrained=True)
        for param in rawnet.features.parameters():
            param.requires_grad = False
```

4. Extract features until the last layer:

```
        self.seq = nn.Sequential(*list(\rawnet.features.children())[:-1])
```

5. Specify that `RoIPool` is to extract a 7 x 7 output. Here, `spatial_scale` is the factor by which proposals (which come from the original image) need to be shrunk so that every output has the same shape before passing through the flatten layer. Images are 224 x 224 in size, while the feature map is 14 x 14 in size:

```
        self.roipool = RoIPool(7, spatial_scale=14/224)
```

6. Define the output heads – `cls_score` and `bbox`:

```
        feature_dim = 512*7*7
        self.cls_score = nn.Linear(feature_dim, len(label2target))
        self.bbox = nn.Sequential(
                        nn.Linear(feature_dim, 512),
                        nn.ReLU(),
                        nn.Linear(512, 4),
                        nn.Tanh(),
                    )
```

7. Define the loss functions:

```
        self.cel = nn.CrossEntropyLoss()
        self.sl1 = nn.L1Loss()
```

8. Define the `forward` method, which takes the image, region proposals, and the index of region proposals as input for the network defined earlier:

```
    def forward(self, input, rois, ridx):
```

9. Pass the `input` image through the pretrained model:

```
        res = input
        res = self.seq(res)
```

10. Create a matrix of `rois` as input for `self.roipool`, first by concatenating `ridx` as the first column and the next four columns being the absolute values of the region proposal bounding boxes:

```
        rois = torch.cat([ridx.unsqueeze(-1), rois*224], dim=-1)
        res = self.roipool(res, rois)
        feat = res.view(len(res), -1)
        cls_score = self.cls_score(feat)
        bbox=self.bbox(feat)#.view(-1,len(label2target),4)
        return cls_score, bbox
```

11. Define the loss value calculation (`calc_loss`), just like we did in the *R-CNN* section:

```
    def calc_loss(self, probs, _deltas, labels, deltas):
        detection_loss = self.cel(probs, labels)
        ixs, = torch.where(labels != background_class)
        _deltas = _deltas[ixs]
        deltas = deltas[ixs]
        self.lmb = 10.0
        if len(ixs) > 0:
            regression_loss = self.sl1(_deltas, deltas)
            return detection_loss + self.lmb * regression_loss, \
                   detection_loss.detach(), regression_loss.detach()
        else:
            regression_loss = 0
```

```
            return detection_loss + self.lmb * regression_loss, \
                   detection_loss.detach(), regression_loss
```

4. Define the functions to train and validate on a batch, just like we did in the *Training R-CNN-based custom object detectors* section:

```
def train_batch(inputs, model, optimizer, criterion):
    input, rois, rixs, clss, deltas = inputs
    model.train()
    optimizer.zero_grad()
    _clss, _deltas = model(input, rois, rixs)
    loss, loc_loss, regr_loss =criterion(_clss,_deltas, clss, deltas)
    accs = clss == decode(_clss)
    loss.backward()
    optimizer.step()
    return loss.detach(), loc_loss, regr_loss, accs.cpu().numpy()
def validate_batch(inputs, model, criterion):
    input, rois, rixs, clss, deltas = inputs
    with torch.no_grad():
        model.eval()
        _clss,_deltas = model(input, rois, rixs)
        loss, loc_loss,regr_loss = criterion(_clss, _deltas, clss, deltas)
        _clss = decode(_clss)
        accs = clss == _clss
    return _clss,_deltas,loss.detach(),loc_loss,regr_loss, accs.cpu().numpy
```

5. Define and train the model over increasing epochs:

```
frcnn = FRCNN().to(device)
criterion = frcnn.calc_loss
optimizer = optim.SGD(frcnn.parameters(), lr=1e-3)
n_epochs = 5
log = Report(n_epochs)
for epoch in range(n_epochs):
    _n = len(train_loader)
    for ix, inputs in enumerate(train_loader):
        loss, loc_loss,regr_loss, accs = train_batch(inputs,
                                frcnn, optimizer, criterion)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, trn_loss=loss.item(),
                   trn_loc_loss=loc_loss,
                   trn_regr_loss=regr_loss,
```

```
                    trn_acc=accs.mean(), end='\r')

    _n = len(test_loader)
    for ix,inputs in enumerate(test_loader):
        _clss, _deltas, loss, \
        loc_loss, regr_loss, accs = validate_batch(inputs,
                                            frcnn, criterion)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, val_loss=loss.item(),
                val_loc_loss=loc_loss,
                val_regr_loss=regr_loss,
                val_acc=accs.mean(), end='\r')
# Plotting training and validation metrics
log.plot_epochs('trn_loss,val_loss'.split(','))
```

The variation in overall loss is as follows:

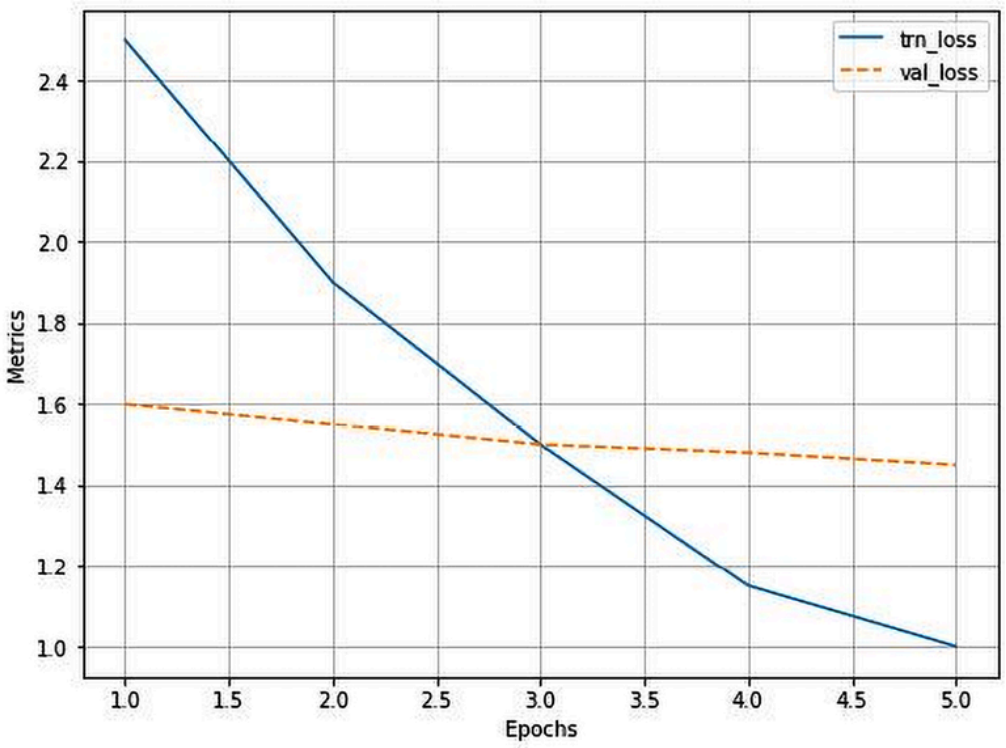


Figure 7.16: Training and validation loss over increasing epochs

6. Define a function to predict on test images:
    1. Define the function that takes a filename as input and then reads the file and resizes it to 224 x 224:

```
import matplotlib.pyplot as plt
%matplotlib inline
import matplotlib.patches as mpatches
from torchvision.ops import nms
from PIL import Image
def test_predictions(filename):
    img = cv2.resize(np.array(Image.open(filename)), (224,224))
```

2. Obtain the region proposals, convert them to the `(x1,y1,x2,y2)` format (top-left pixel and bottom-right pixel coordinates), and then convert these values to the ratio of width and height that they are present in, in proportion to the image:

```
    candidates = extract_candidates(img)
    candidates = [(x,y,x+w,y+h) for x,y,w,h in candidates]
```

3. Preprocess the image and scale the RoIs (`rois`):

```
    input = preprocess_image(img/255.)[None]
    rois = [[x/224,y/224,X/224,Y/224] for x,y,X,Y in candidates]
```

4. As all proposals belong to the same image, `rixs` will be a list of zeros (as many as the number of proposals):

```
    rixs = np.array([0]*len(rois))
```

5. Forward-propagate the input, RoIs through the trained model, and get the confidence and class scores for each proposal:

```
    rois,rixs = [torch.Tensor(item).to(device) for item in [rois, rixs]]
    with torch.no_grad():
        frcnn.eval()
        probs, deltas = frcnn(input, rois, rixs)
        confs, clss = torch.max(probs, -1)
```

6. Filter out the background class:

```
    candidates = np.array(candidates)
    confs,clss,probs,deltas=[tensor.detach().cpu().numpy() \
             for tensor in [confs, clss, probs, deltas]]

    ixs = clss!=background_class
    confs, clss, probs,deltas,candidates= [tensor[ixs] for \
          tensor in [confs, clss, probs, deltas,candidates]]
    bbs = candidates + deltas
```

7. Remove near-duplicate bounding boxes with `nms`, and get indices of those proposals in which the highly confident models are objects:

```
    ixs = nms(torch.tensor(bbs.astype(np.float32)),
                torch.tensor(confs), 0.05)
    confs, clss, probs,deltas,candidates,bbs= [tensor[ixs] \
                  for tensor in [confs,clss,probs, deltas, candidates, bbs]
    if len(ixs) == 1:
        confs, clss, probs, deltas, candidates, bbs = \
                    [tensor[None] for tensor in [confs,clss,
                            probs, deltas, candidates, bbs]]

    bbs = bbs.astype(np.uint16)
```

8. Plot the bounding boxes obtained:

```
    _, ax = plt.subplots(1, 2, figsize=(20,10))
    show(img, ax=ax[0])
    ax[0].grid(False)
    ax[0].set_title(filename.split('/')[-1])
    if len(confs) == 0:
        ax[1].imshow(img)
        ax[1].set_title('No objects')
        plt.show()
        return
    else:
        show(img,bbs=bbs.tolist(),
    texts=[target2label[c] for c in clss.tolist()],ax=ax[1])
        plt.show()
```

7. Predict on a test image:

```
test_predictions(test_ds[29][-1])
```

The preceding code results in the following:

Figure 7.17: Original image and the predicted bounding box and classes

The preceding code executes in 1.5 seconds. This is primarily because we are still using two different models, one to generate region proposals and another to make predictions of class and corrections. In the next chapter, we will learn about having a single model to make predictions so that inference is quick in a real-time scenario.

Summary

In this chapter, we began by learning about creating a training dataset for the process of object localization and detection. Then, we learned about SelectiveSearch, a region proposal technique that recommends regions based on the similarity of pixels in proximity. We also learned about calculating the IoU metric to understand the goodness of the predicted bounding box around the objects present in the image.

In addition, we looked at performing non-max suppression to fetch one bounding box per object within an image, before learning about building R-CNN and Fast R-CNN models from scratch. We also explored why R-CNN is slow and how Fast R-CNN leverages RoI pooling and fetches region proposals from feature maps to make inference faster. Finally, we under-

stood that having region proposals coming from a separate model results in more time taken to predict on new images.

In the next chapter, we will learn about some of the modern object detection techniques that are used to make inferences on a more real-time basis.

Questions

1. How does the region proposal technique generate proposals?
2. How is IoU calculated if there are multiple objects in an image?
3. Why is Fast R-CNN faster than R-CNN?
4. How does RoI pooling work?
5. What is the impact of not having multiple layers after obtaining a feature map when predicting bounding-box corrections?
6. Why do we have to assign a higher weight to regression loss when calculating overall loss?
7. How does non-max suppression work?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/modcv